PageRank: Closed-Form Derivation, Implementation, and Application to AI Web Crawl Prioritization

Au Ze Hong, Maxwell
U2321963E

1 Setup

Consider a web graph with N pages. Define:

- A = adjacency matrix, where $A_{ij} = 1$ if page i links to page j, else 0.
- H = row-stochastic transition matrix, where $H_{ij} = \frac{1}{\text{out-degree}(i)}$ if $i \to j$, else 0.
- p = teleport probability, $0 < p \leq 1$.
- $E = N \times N$ all-ones matrix.
- $\mathbf{1}$ = all-ones column vector of length N.

The Google transition matrix is:

$$M = (1-p)H + \frac{p}{N}E \tag{1}$$

2 Steady-State Condition

The PageRank vector $\mathbf{r}^T$ is the stationary distribution of the Markov chain defined by M, satisfying:

$$\mathbf{r}^T M = \mathbf{r}^T \tag{2}$$

with the constraint $\mathbf{r}^T \mathbf{1} = 1$ (i.e., $\mathbf{r}$ is a probability distribution).

3 Closed-Form Derivation

Substituting the expression for M:

$$\mathbf{r}^T \left[(1-p)H + \frac{p}{N}E\right] = \mathbf{r}^T \tag{3}$$

Expanding:

$$(1-p)\mathbf{r}^T H + \frac{p}{N}\mathbf{r}^T E = \mathbf{r}^T \tag{4}$$

Since $\mathbf{r}$ is a probability distribution, $\mathbf{r}^T\mathbf{1} = 1$. Noting that $E = \mathbf{1}\mathbf{1}^T$, we have:

$$\mathbf{r}^T E = \mathbf{r}^T\mathbf{1}\mathbf{1}^T = 1 \cdot \mathbf{1}^T = \mathbf{1}^T \tag{5}$$

Substituting:

$$(1-p)\mathbf{r}^T H + \frac{p}{N}\mathbf{1}^T = \mathbf{r}^T \tag{6}$$

Rearranging:

$$\mathbf{r}^T - (1-p)\mathbf{r}^T H = \frac{p}{N}\mathbf{1}^T \tag{7}$$

Factoring:

$$\mathbf{r}^T \left[I - (1-p)H\right] = \frac{p}{N}\mathbf{1}^T \tag{8}$$

Solving for $\mathbf{r}^T$:

$$\boxed{\mathbf{r}^T = \frac{p}{N} \cdot \mathbf{1}^T \cdot \left[I - (1-p)H\right]^{-1}} \tag{9}$$

4 Invertibility of $I - (1-p)H$

Since H is row-stochastic, its eigenvalues satisfy $|\lambda_i| \leq 1$. For $0 < p \leq 1$, the matrix $(1-p)H$ has spectral radius:

$$\rho\big((1-p)H\big) \leq (1-p) < 1 \tag{10}$$

Therefore $I - (1-p)H$ is non-singular and its inverse exists. Moreover, it admits a Neumann series expansion:

$$\left[I - (1-p)H\right]^{-1} = \sum_{k=0}^{\infty} \left[(1-p)H\right]^k = I + (1-p)H + (1-p)^2 H^2 + \cdots \tag{11}$$

This series converges absolutely since $(1-p) < 1$, and corresponds to the iterative power method: each term $(1-p)^k H^k$ represents the contribution of k-step random walks, geometrically decayed.

5 Effect of p

- As $p \to 0$: $M \to H$. The PageRank is determined entirely by the link structure. Rank concentrates on highly connected pages.
- At $p = 0.15$: The standard value used by Google. Link structure dominates with mild smoothing from teleportation.
- As $p \to 1$: $M \to \frac{1}{N}E$. The surfer teleports at every step, so $\mathbf{r} \to \frac{1}{N}\mathbf{1}$. All pages receive equal rank regardless of link structure.

We verify this experimentally on the 10,000-node Google web graph. Table 1 shows the top-ranked node's PageRank score at varying p values. At $p = 0.01$, the top node (486980) holds rank 263× the uniform value $1/N = 10^{-4}$. As p increases, this concentration diminishes: at $p = 0.85$, the top score is only 12.6× uniform, and the identity of the top node changes (from 486980 to 151110), reflecting the erosion of link-based authority. Figure 1 visualizes this convergence toward uniformity for the top-5 nodes.

p	**Top Node**	**Score**	**Ratio to** $1/N$
0.01	486980	0.02629	262.9×
0.05	486980	0.01225	122.5×
0.15	486980	0.00700	70.0×
0.50	486980	0.00313	31.3×
0.85	151110	0.00126	12.6×
0.95	151110	0.00053	5.3×

Table 1: Top-ranked node's PageRank score vs. teleport probability p on web-Google_10k.txt ($N = 10{,}000$, uniform $= 10^{-4}$)

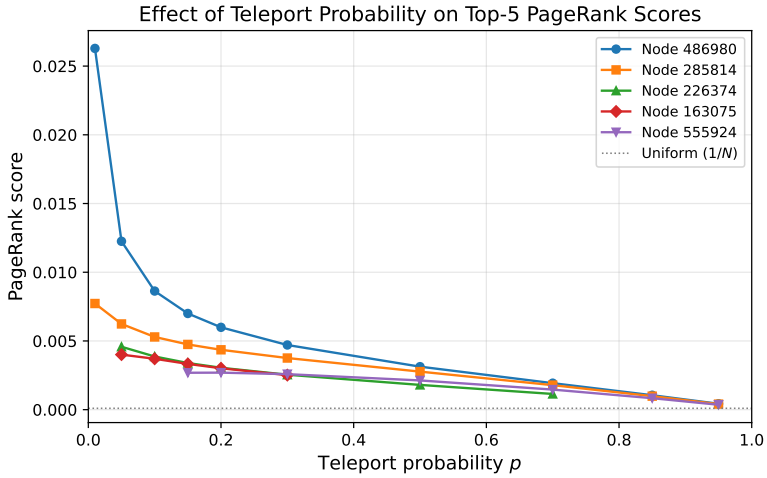

Figure 1: PageRank scores of top-5 nodes vs. teleport probability p on web-Google_10k.txt. As $p \to 1$, all scores converge toward the uniform baseline $1/N$.

6 Illustrative Example

Consider $N = 3$ pages with links $1 \to 2$, $2 \to 1$, $2 \to 3$, $3 \to 2$, and $p = 0.5$.

The row-stochastic transition matrix is:

$$H = \begin{pmatrix} 0 & 1 & 0 \\ \frac{1}{2} & 0 & \frac{1}{2} \\ 0 & 1 & 0 \end{pmatrix} \tag{12}$$

Computing:

$$I - (1-0.5)H = \begin{pmatrix} 1 & -\frac{1}{2} & 0 \\ -\frac{1}{4} & 1 & -\frac{1}{4} \\ 0 & -\frac{1}{2} & 1 \end{pmatrix} \tag{13}$$

Applying the closed form, we first compute the inverse. The determinant is:

$$\det\left(I - \tfrac{1}{2}H\right) = 1\cdot\left(1 - \tfrac{1}{8}\right) - \left(-\tfrac{1}{2}\right)\cdot\left(-\tfrac{1}{4}\right) = \tfrac{7}{8} - \tfrac{1}{8} = \tfrac{3}{4} \tag{14}$$

Computing the adjugate and dividing by the determinant:

$$\left[I - \tfrac{1}{2}H\right]^{-1} = \frac{4}{3}\begin{pmatrix} \frac{7}{8} & \frac{1}{2} & \frac{1}{8} \\ \frac{1}{4} & 1 & \frac{1}{4} \\ \frac{1}{8} & \frac{1}{2} & \frac{7}{8} \end{pmatrix} = \begin{pmatrix} \frac{7}{6} & \frac{2}{3} & \frac{1}{6} \\ \frac{1}{3} & \frac{4}{3} & \frac{1}{3} \\ \frac{1}{6} & \frac{2}{3} & \frac{7}{6} \end{pmatrix} \tag{15}$$

Then:

$$\mathbf{1}^T \cdot \left[I - \tfrac{1}{2}H\right]^{-1} = \left(\tfrac{7}{6} + \tfrac{1}{3} + \tfrac{1}{6},\ \tfrac{2}{3} + \tfrac{4}{3} + \tfrac{2}{3},\ \tfrac{1}{6} + \tfrac{1}{3} + \tfrac{7}{6}\right) = \left(\tfrac{5}{3},\ \tfrac{8}{3},\ \tfrac{5}{3}\right) \tag{16}$$

Finally:

$$\mathbf{r}^T = \frac{0.5}{3} \cdot \left(\tfrac{5}{3},\ \tfrac{8}{3},\ \tfrac{5}{3}\right) = \left(\tfrac{5}{18},\ \tfrac{4}{9},\ \tfrac{5}{18}\right) \approx (0.2778,\ 0.4444,\ 0.2778) \tag{17}$$

We verify $\sum_i r_i = \frac{5}{18} + \frac{4}{9} + \frac{5}{18} = 1$. Page 2 has the highest rank ($r_2 = 4/9$) because it receives links from both pages 1 and 3, while pages 1 and 3 each receive a link only from page 2 and thus share equal rank. This matches the result obtained by iterating $\mathbf{r}^T \leftarrow \mathbf{r}^T M$ until convergence.

7 Implementation

We implement both the closed-form solution and the iterative power method in Go (using the `gonum` library for dense linear algebra), and evaluate them on two Google web graph datasets. All experiments were run on a single machine; the source code is available in the repository listed in Section 9.

7.1 Iterative Method (Power Method)

The power method computes PageRank by repeatedly applying the transition:

$$\mathbf{r} \leftarrow (1-p)H^T\mathbf{r} + \frac{p}{N}\mathbf{1} \tag{18}$$

starting from $\mathbf{r} = \frac{1}{N}\mathbf{1}$, until the L_1 norm of the change $\|\mathbf{r}^{(t+1)} - \mathbf{r}^{(t)}\|_1 < \epsilon$. This method operates directly on the sparse adjacency list, requiring only $O(N+|E|)$ memory and $O(|E|)$ work per iteration, where $|E|$ is the number of edges.

Dangling nodes (pages with no outlinks) are handled by redistributing their rank uniformly across all pages at each iteration. Figure 2 shows the convergence behavior on the 10,000-node dataset: the L_1 difference decreases exponentially, reaching $\epsilon = 10^{-10}$ in 114 iterations.

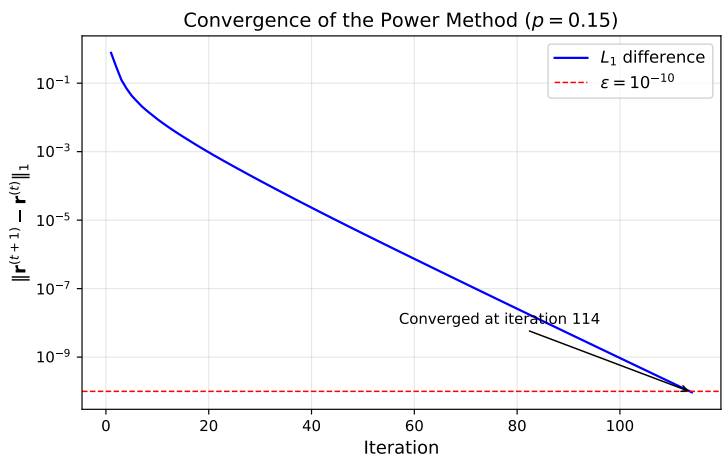

Figure 2: Convergence of the power method on web-Google_10k.txt ($p = 0.15$). The L_1 norm of the rank difference decreases exponentially, converging in 114 iterations.

7.2 Closed-Form Method (Matrix Inversion)

The closed-form solution requires solving the linear system:

$$[I - (1-p)H]^T \mathbf{r} = \frac{p}{N}\mathbf{1} \tag{19}$$

This requires constructing the full $N \times N$ dense matrix $I - (1-p)H$ and performing LU decomposition to solve the system.

7.3 Results on web-Google_10k.txt

On the 10,000-node subgraph (78,323 edges), both methods produce identical results with a maximum absolute difference of 7.67×10^{-12}, confirming correctness.

However, the two methods differ dramatically in performance:

The closed-form method is approximately 382× slower than the iterative method on this dataset.

Rank	Node ID	Score
1	486980	0.0069990194
2	285814	0.0047475463
3	226374	0.0033955805
4	163075	0.0033308254
5	555924	0.0026860608

Table 2: Top 5 pages by PageRank on web-Google_10k.txt ($p = 0.15$)

Method	Time	Memory	Complexity
Iterative (power method)	25 ms	$O(N + \|E\|)$	$O(\text{iters} \times \|E\|)$
Closed-form (LU solve)	9.7 s	$O(N^2)$	$O(N^3)$

Table 3: Performance comparison on web-Google_10k.txt ($N = 10{,}000$)

7.4 Results on web-Google.txt

On the full Google web graph ($N = 875{,}713$ nodes, $|E| = 5{,}105{,}039$ edges), the iterative method converges in 118 iterations and 3.1 seconds.

Rank	Node ID	Score
1	597621	0.0009237249
2	41909	0.0009211312
3	163075	0.0009040044
4	537039	0.0008988318
5	384666	0.0007868924

Table 4: Top 5 pages by PageRank on web-Google.txt ($p = 0.15$)

The closed-form method cannot be applied to this dataset: constructing the dense $875{,}713 \times 875{,}713$ matrix requires $875{,}713^2 \times 8$ bytes ≈ 5.7 TB of memory, which exceeds available system RAM by orders of magnitude. The program terminates with an out-of-memory error before the solve can begin.

This demonstrates the fundamental advantage of the iterative power method: it operates on the **sparse** graph structure directly, requiring memory proportional to the number of edges rather than the square of the number of nodes. For real-world web graphs with millions or billions of pages, the closed-form solution is entirely infeasible, while the iterative method remains practical.

8 AI Web Crawl Prioritization

We extend the PageRank implementation to design a crawling strategy for an AI research organization collecting training data, similar to OpenAI's GPTBot. The program takes a directed web graph (represented as a JSON dictionary of URLs and their outlinks) along with precomputed PageRank scores, and returns the top-k URLs to crawl.

8.1 Why High-PageRank Pages Yield Better Training Data

PageRank measures recursive authority: a page is important if many important pages link to it. For generative AI training, prioritizing high-PageRank pages is beneficial because:

1. **Factual accuracy**: Widely-cited pages undergo more scrutiny and correction over time, making their content more reliable. Training on authoritative text reduces hallucination risk.
2. **Topical breadth**: High-PageRank pages tend to serve as reference material for many other sites, covering topics broadly and clearly.
3. **Content structure**: Authoritative pages are often well-structured with headings, paragraphs, and citations, making them easier to parse into clean training data.
4. **Diversity of knowledge**: Since PageRank propagates through the link graph, highly-ranked pages span diverse domains (encyclopedias, government databases, academic repositories), providing a natural curriculum for training.

8.2 Heuristic: Authority-Demand-Diversity-Trust (ADDT) Score

We propose a multi-factor heuristic that combines four signals to identify high-quality pages that permit crawling:

$$\text{score}(v) = w_1 \cdot \widehat{\text{PR}}(v) + w_2 \cdot \widehat{\text{InDeg}}(v) + w_3 \cdot \widehat{\text{OutDeg}}(v) + w_4 \cdot \text{Trust}(v) \quad (20)$$

where $\widehat{\cdot}$ denotes min-max normalization to $[0, 1]$ across all crawlable pages, and:

- **Authority** ($w_1 = 0.40$): Normalized PageRank score, capturing recursive trust from the web graph.
- **Demand** ($w_2 = 0.20$): Normalized in-degree, measuring how many pages link to this page as an independent validation signal distinct from PageRank's recursive propagation.
- **Diversity** ($w_3 = 0.20$): Normalized out-degree, favoring pages with broad outlinks that serve as good entry points for expanding the crawl frontier.
- **Domain Trust** ($w_4 = 0.20$): A TLD-based prior reflecting content quality expectations:
  - `.edu`, `.gov` $\rightarrow 1.0$ (academic/government, highest factual standards)
  - `.org` $\rightarrow 0.7$ (non-profit, generally reliable)

- `.io` → 0.4 (tech-focused)
- Other → 0.3 (commercial/unknown)

Before scoring, pages that disallow crawling are filtered out by checking `robots.txt` for the target bot name (e.g., `GPTBot`). The program parses `User-agent` and `Disallow` directives, respecting both bot-specific and wildcard rules. Pages whose `robots.txt` is unreachable are assumed to permit crawling (fail-open).

8.3 Input Format

The program accepts a JSON dictionary where keys are URLs and values are lists of outlinks:

```
{
  "https://en.wikipedia.org": ["https://www.nist.gov", "https://archive.org"],
  "https://www.nist.gov":     ["https://en.wikipedia.org", "https://arxiv.org"],
  ...
}
```

This graph is parsed into the same internal representation used for PageRank computation. URLs appearing only as outlinks (not as keys) are added as nodes with zero out-degree.

8.4 Demo Results

We evaluate on a 10-node graph with realistic URLs spanning `.org`, `.edu`, `.gov`, `.com`, and `.io` domains. Amazon and Facebook are simulated as blocking GPT-Bot via `robots.txt`.

Rank	URL	PageRank	InDeg	OutDeg	Trust	Score
1	en.wikipedia.org	0.2260	6	3	.org	0.890
2	www.nist.gov	0.1265	3	3	.gov	0.674
3	www.example.com	0.1647	5	3	.com	0.668
4	archive.org	0.1181	3	2	.org	0.549
5	arxiv.org	0.1042	3	2	.org	0.524

Table 5: Top 5 crawl targets by ADDT score (weights: $w_1 = 0.40, w_2 = 0.20, w_3 = 0.20, w_4 = 0.20$)

Wikipedia ranks first due to the highest PageRank (most pages link to it) combined with broad outlinks and a trusted `.org` domain. NIST outranks the higher-PageRank `example.com` because its `.gov` domain trust (1.0 vs. 0.3) compensates for the PageRank difference, demonstrating how the multi-factor heuristic surfaces high-quality government sources that pure PageRank ranking alone might deprioritize. Amazon and Facebook are excluded entirely due to their `robots.txt` restrictions. Figure 3 visualizes the score breakdown.

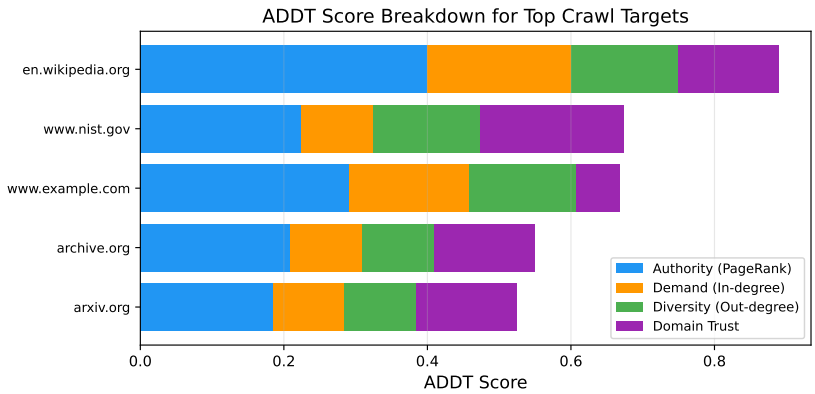

Figure 3: ADDT score breakdown for top-5 crawl targets. Each bar segment shows the weighted contribution of authority (PageRank), demand (in-degree), diversity (out-degree), and domain trust.

8.5 Application to the Google Web Graph

When applied to the 10,000-node Google dataset (where nodes are integer IDs without domain information), the domain trust factor is automatically disabled and its weight redistributed to authority. The top crawl target is node 285814, which balances high PageRank (rank 2 overall) with the highest in-degree (207) and out-degree (210) in the graph, making it an ideal crawl frontier entry point. Node 486980, despite having the highest PageRank, ranks second for crawling because its low out-degree (6) limits its value as a source for discovering additional pages.

9 Code

The full source code is available at: `https://github.com/maxwellau2/cloud-assignment-2`